

Terraform Lifecycle Lab Guide

Instructor-led practice: init, validate, plan, apply, destroy, and core .tf blocks

1. Learning Objectives

- Explain Terraform's declarative workflow and the role of state.
- Run **terraform init**, **validate**, **plan**, **apply**, and **destroy** safely.
- Read and write the main blocks found in Terraform configuration files.
- Interpret command output, verify created resources, and clean up the lab environment.

2. Lab Overview and Prerequisites

Audience: Beginners with basic command-line familiarity. **Estimated time:** 45–60 minutes. This lab uses HashiCorp's **local** provider, so it creates only a text file on the learner's computer and requires no cloud account.

- Terraform CLI installed and available in the terminal.
- A text editor and permission to create files in a working folder.
- Internet access during **terraform init** to download the provider plugin.
- Verify installation with: **terraform version**.
- Do not commit secrets, credentials, the **.terraform** directory, or sensitive state files to source control.

3. Build the Lab Configuration

Command convention: Text beginning with **terraform**, **mkdir**, **cd**, or **code** is entered in the terminal. Terraform configuration beginning with blocks such as **terraform**, **variable**, **resource**, **data**, or **output** belongs in a **.tf** file. Do not copy explanatory labels such as **Checkpoint** or **Expected result** into the terminal.

1. Create and enter the lab folder by copying the commands for your terminal.
Windows PowerShell: **mkdir terraform-lifecycle-lab; cd terraform-lifecycle-lab**.
macOS or Linux: **mkdir terraform-lifecycle-lab && cd terraform-lifecycle-lab**.
Confirm that the terminal prompt is inside this folder before creating **main.tf**.
2. Open a terminal in that folder.
3. Create a file named **main.tf** in the **terraform-lifecycle-lab** folder. In Visual Studio Code, copy and run **code main.tf**. If the **code** command is unavailable, create the file with any plain-text editor. Paste the complete configuration shown below, save the file, and return to the terminal.
4. Enter the configuration below and save the file.

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

variable "student_name" {
  description = "Name placed in the generated file"
  type        = string
  default     = "Terraform Student"
  validation {
    condition     = length(trimspaces(var.student_name)) > 0
    error_message = "student_name must not be empty."
  }
}

locals {
  lab_message = "Hello, ${var.student_name}! Terraform created this
file."
}

resource "local_file" "lab_note" {
  filename = "${path.module}/lab-output.txt"
  content  = local.lab_message

  lifecycle {
    prevent_destroy = false
  }
}

data "local_file" "lab_note_readback" {
  filename = local_file.lab_note.filename
  depends_on = [local_file.lab_note]
}

output "created_file" {
  description = "Absolute path of the generated file"
  value       = local_file.lab_note.filename
}

```

```
output "file_contents" {
  description = "Content read back through a data source"
  value       = data.local_file.lab_note_readback.content
}
```

4. Understanding the Blocks in a .tf File

Terraform reads all **.tf** files in the current directory as one root module. A block has a type, optional labels, and a body. Arguments assign values, expressions calculate values, and references create dependencies. File names such as **main.tf**, **variables.tf**, and **outputs.tf** are conventions for readability; Terraform combines them automatically.

Block	Purpose	Example or key idea
terraform	Configures Terraform itself.	required_version , provider requirements, and optionally a backend.
required_providers	Declares provider source addresses and acceptable versions.	local = { source = "hashicorp/local", version = "~> 2.5" }
provider	Configures a provider plugin.	provider "local" {} ; cloud providers commonly take region or subscription settings.
variable	Defines an input with type, default, description, sensitivity, or validation.	Reference with var.student_name .
locals	Defines reusable expressions internal to the module.	Reference with local.lab_message .
resource	Declares infrastructure Terraform creates or manages.	resource "local_file" "lab_note" ; reference as local_file.lab_note .
data	Reads existing information without owning its lifecycle.	data "local_file" "lab_note_readback" .
output	Exposes useful values after an apply.	View with terraform output .

module	Calls a reusable collection of Terraform files.	module "example" { source = "./modules/example" }
lifecycle	Changes how Terraform handles a resource.	Supports rules such as create_before_destroy , prevent_destroy , and ignore_changes .
moved / import / check	Supports refactoring, adopting existing objects, and assertions.	Use when the exercise advances beyond basic creation.

4.1 Block Explanations and Optional Examples

terraform and required_providers: The top-level **terraform** block constrains the Terraform CLI version and declares provider dependencies. The nested **required_providers** block tells **init** where to obtain each plugin and which versions are compatible. Provider version selections are recorded in **.terraform.lock.hcl**; commit that lock file for repeatable team runs.

provider: A provider block supplies provider-specific configuration. For example, an AWS configuration might be **provider "aws" { region = "ap-south-1" }**. Prefer environment variables, workload identities, or a secure secret store for credentials rather than writing secrets in a **.tf** file.

variable and locals: Input variables make a module configurable and can be populated by a default, a **.tfvars** file, an environment variable, or **-var**. A validation block rejects unsuitable values. Locals reduce repetition but are not set directly by callers. Example override: **terraform plan -var="student_name=Asha"**.

resource and data: A resource block declares an object Terraform owns and tracks in state. Its two labels identify the resource type and local name. A data block reads information; it does not create the underlying object. Referencing **local_file.lab_note.filename** creates an implicit dependency. Use **depends_on** only when a real dependency is not otherwise visible in an expression.

output and module: Output blocks display or export selected values. Mark confidential values with **sensitive = true**, while remembering that sensitive values may still exist in state. A module block calls reusable Terraform code, for example: **module "web" { source = "./modules/web" instance_count = 2 }**. Inputs are arguments to the module, and the caller can read values such as **module.web.endpoint**.

lifecycle meta-argument: This nested resource block is different from the CLI lifecycle in Section 5. **create_before_destroy = true** asks Terraform to create a replacement before removing the old object when possible. **prevent_destroy = true** rejects plans that would destroy the resource until the rule is removed. **ignore_changes = [attribute]** tells Terraform not to reconcile selected post-creation attribute changes. Use these rules deliberately because they alter normal planning behavior.

backend and advanced top-level blocks: A nested **backend** block in **terraform** selects where state is stored; backend configuration is initialized with **terraform init**, and values generally cannot refer to variables. A **moved** block records a resource-address refactor, an **import** block brings an existing object under management during apply, and a **check** block expresses assertions. These are useful after learners master the basic workflow.

5. Step-by-Step Terraform CLI Lifecycle

Student quick start: Complete the commands in order. Copy only the command shown in each command box, paste it into a terminal, press Enter, and compare the result with the checkpoint before continuing. Run every command from the **terraform-lifecycle-lab** folder. If a command fails, stop and use the troubleshooting section before proceeding.

Run each command from the folder containing **main.tf**. The safe sequence is: initialize, format and validate, preview, apply the reviewed plan, inspect, modify and re-plan, and finally destroy.

Step 1 — Initialize: **terraform init**

1. Run **terraform init**.
2. Observe that Terraform initializes the working directory, installs the **hashicorp/local** provider, and writes or updates **.terraform.lock.hcl**.
3. If provider requirements or backend settings later change, run **terraform init** again.

Checkpoint: Successful initialization ends with **Terraform has been successfully initialized!** The command also creates the hidden **.terraform** directory and writes or updates **.terraform.lock.hcl**. Do not edit files inside **.terraform**.

Step 2 — Format and Validate: **terraform fmt**, **terraform validate**

1. Copy and run these commands one at a time: **terraform fmt**; **terraform validate**. Continue only after Terraform reports that the configuration is valid. For an optional troubleshooting exercise, remove one closing brace from **main.tf**, save the file, run **terraform validate**, read the diagnostic, restore the brace, save, and validate again.

Checkpoint: Look for a success message indicating that the configuration is valid. Validation does not prove that every remote operation will succeed; permissions, quotas, and runtime API conditions are evaluated later.

Step 3 — Preview: terraform plan

1. Copy and run: **terraform plan -out=tfplan**. Do not apply yet. Review the action symbols and confirm that Terraform plans to create the local file. Then copy and run: **terraform show tfplan**. Continue only if the saved plan contains the expected changes and no unexpected deletion.
2. Confirm that the plan proposes creation of **local_file.lab_note**. The **local_file** data source reads the file after the resource is created because its filename depends on the resource, so its value may be shown as known after apply rather than as a separately created object.

Checkpoint: A plan previews changes but does not create the file. Saving the plan allows the next command to apply exactly the reviewed actions. Treat saved plans as potentially sensitive artifacts.

Step 4 — Create or Update: terraform apply

1. Run **terraform apply tfplan**. Terraform applies the previously saved and reviewed plan without asking for an additional approval prompt. Use a saved plan only when its contents have been reviewed and the configuration and state have not changed. Then run **terraform output** and **terraform plan**. Open **lab-output.txt** and verify that it contains the greeting. The final plan should report that no changes are required.

Checkpoint: Terraform creates **terraform.tfstate**, which maps configuration addresses to managed objects. Do not edit state manually, and protect it because it can contain sensitive values.

Step 5 — Change and Reconcile

1. Choose your name and replace **YourName** in both commands before running them. Example: **terraform plan -var="student_name=Asha"**. If the plan shows only the expected content update, run **terraform apply -var="student_name=Asha"**, review the plan, and enter **yes**. Open **lab-output.txt** and confirm that the greeting now contains Asha.
2. For repeatable input, create **terraform.tfvars** containing **student_name = "YourName"** instead of repeatedly using **-var**.

Step 6 — Remove Managed Objects: terraform destroy

1. Copy and run: **terraform plan -destroy**. Verify that the plan removes only **lab-output.txt** and its managed resource. Then copy and run: **terraform destroy**. Review the proposed actions, enter **yes**, and confirm that **lab-output.txt** no longer exists. If you created **terraform.tfvars**, Terraform will automatically reuse that value.

Important: terraform destroy removes resources tracked by the current state and workspace; it does not delete the Terraform configuration itself. In shared or production environments, require plan review and appropriate authorization before applying destructive actions.

6. Lifecycle Command Summary

Student progress checklist: Terraform version displayed; lab folder created; **main.tf** saved; initialization successful; configuration valid; plan reviewed and saved; plan applied; output file verified; second plan reported no changes; variable update applied; destroy plan reviewed; resource destroyed.

Command	Question it answers	Main result
terraform init	Is the working directory ready?	Initializes backend, providers, modules, and dependency lock selections.
terraform validate	Is the configuration internally valid?	Reports syntax, schema, type, and reference problems that can be checked statically.
terraform plan	What would Terraform change?	Compares configuration, state, and managed objects and produces proposed actions.
terraform apply	Should the reviewed changes be executed?	Creates, updates, replaces, or deletes objects and updates state.
terraform destroy	Should all objects managed by this configuration and workspace be removed?	Plans and applies removal actions after confirmation.

7. Troubleshooting Guide

- **terraform command not found:** Install Terraform and ensure its executable directory is on the system path; reopen the terminal.

- **Provider download fails:** Check network or proxy access, then retry **terraform init**.
- **Validation reports an unsupported argument:** Check spelling and the installed provider schema; run **terraform init** after changing provider requirements.
- **Saved plan is stale:** Create a new plan if configuration, variables, or state changed after **tfplan** was saved.
- **State is locked:** Wait for another run to finish. Do not force-unlock unless the lock is known to be abandoned and the lock identifier is verified.
- **Destroy is blocked:** Check for **prevent_destroy = true**; remove it only after confirming the deletion is intentional.

8. Review Questions and Instructor Checklist

1. Why must **terraform init** usually run before validation or planning?
2. What is the difference between a provider requirement and a provider configuration?
3. Why is **terraform plan** safer than applying immediately?
4. How does a resource differ from a data source?
5. What creates the dependency between the data source and the local file in this lab?
6. When would **prevent_destroy** be appropriate, and what operational risk can it introduce?
7. Why should a team protect state and retain **.terraform.lock.hcl**?

Completion criteria: The learner can explain the purpose of each core block; run **terraform init**, **validate**, **plan**, **apply**, and **destroy** in the correct order; identify create, update, replace, and destroy actions in a plan; verify the generated file and outputs; produce a no-change plan; and confirm that destroy removes the managed file.

9. Cleanup and Optional Extension

After confirming destruction, learners may delete the disposable lab folder. For an extension, split the configuration into **versions.tf**, **variables.tf**, **main.tf**, and **outputs.tf**, then rerun the lifecycle to demonstrate that file organization does not change Terraform's module behavior.